

**POKHARA UNIVERSITY
GANDAKI COLLEGE OF ENGINEERING AND
SCIENCE**

LABORATORY REPORT



AGILE SOFTWARE DEVELOPMENT LAB

Lab No. 5

BESE III YEAR - VI SEM

Submitted By:

Sudip Sharma

Roll No: 39

Program: BESE ,2023

Submitted To:

Er. Rajendra Bahadur Thapa

Lecturer

Date: July 3, 2026

NAME OF EXPERIMENT

Implementation of CI/CD Pipeline Using GitHub Actions and Exploration of Jenkins.

AIM

To implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline using GitHub Actions and study the features and working of Jenkins as a CI/CD automation tool.

THEORY

Continuous Integration (CI) and **Continuous Deployment (CD)** are fundamental DevOps practices that automate the software delivery lifecycle. They enable development teams to integrate code changes frequently, run automated tests, and deploy applications reliably and rapidly. CI/CD pipelines reduce manual intervention, minimize human errors, and accelerate the feedback loop between development and operations teams.

1. **Continuous Integration (CI):** Continuous Integration is the practice of automatically building and testing code whenever changes are pushed to a shared version control repository. Developers are encouraged to commit code frequently—often multiple times per day. Each commit triggers an automated build and test process, ensuring that the new code integrates smoothly with the existing codebase.

Key Benefits of CI:

- Early detection of integration bugs and conflicts.
- Automated testing ensures that new changes do not break existing functionality.
- Faster development cycles with immediate feedback to developers.
- Improved code quality through consistent testing and validation.
- Reduced manual effort in building and testing code.

2. **Continuous Deployment (CD):** Continuous Deployment extends CI by automatically deploying the application to staging or production environments after it successfully passes all automated tests. This eliminates the need for manual deployment processes, allowing teams to deliver new features, bug fixes, and updates to end-users rapidly and reliably.

Key Benefits of CD:

- Faster and more frequent software releases.
- Reduced deployment errors through automation.
- Continuous delivery of new features and improvements.
- Improved application reliability through consistent deployment processes.
- Enhanced ability to respond to customer feedback quickly.

3. **GitHub Actions:** GitHub Actions is a powerful CI/CD platform built directly into GitHub. It allows developers to automate workflows directly from their repositories without requiring external tools. Workflows are defined using YAML files and can be triggered by various GitHub events such as pushes, pull requests, releases, or scheduled intervals.

Key Features of GitHub Actions:

- *Workflow Automation:* Define custom workflows for building, testing, and deploying applications.
- *Actions Marketplace:* Pre-built actions for common tasks (e.g., checking out code, setting up environments, deploying to cloud platforms).
- *Matrix Builds:* Test across multiple operating systems, languages, and versions simultaneously.
- *Event-Driven:* Trigger workflows based on GitHub events.
- *Cross-Platform Support:* Run workflows on Windows, Linux, and macOS runners.
- *Secrets Management:* Securely store and use sensitive information such as API keys and tokens.

4. **Jenkins:** Jenkins is an open-source automation server that has been a cornerstone of CI/CD for over a decade. It is highly extensible through its vast plugin ecosystem, which includes integrations with Git, Docker, Kubernetes, cloud platforms, and numerous testing tools. Jenkins supports both Declarative and Scripted Pipelines, enabling teams to define their build, test, and deployment processes as code.

Key Features of Jenkins:

- *Open-Source and Free:* No licensing costs, with a large active community.
- *Pipeline as Code:* Define pipelines using Groovy-based DSL (Domain Specific Language).
- *Plugin Ecosystem:* Over 1,800 plugins available for integrating with almost any tool or platform.
- *Distributed Builds:* Distribute build and test workloads across multiple agent nodes.

- *Automated Testing*: Integrate with testing frameworks to run unit, integration, and functional tests.
- *Integration Capabilities*: Seamlessly integrates with Git, Docker, Kubernetes, AWS, Azure, and more.
- *Web Interface*: User-friendly dashboard for monitoring builds, viewing logs, and managing jobs.

Together, CI/CD practices and tools like GitHub Actions and Jenkins enable teams to adopt DevOps principles, improve collaboration between developers and operations, and deliver high-quality software at speed.

OBJECTIVES

- To understand the fundamental concepts of Continuous Integration and Continuous Deployment (CI/CD).
- To implement a functional CI/CD pipeline using GitHub Actions for automated build and testing.
- To automate project builds and test execution using YAML-defined workflows.
- To explore workflow automation triggered by repository events (e.g., push).
- To study the architecture, features, and capabilities of Jenkins as a CI/CD automation server.
- To understand the role of automation in modern Agile and DevOps software development.

ALGORITHM

1. Create a GitHub repository for the sample project.
2. Develop a simple application (e.g., a Node.js or Python application) and push it to the repository.
3. Ensure the application has a test suite (e.g., Jest for JavaScript or pytest for Python) that can be executed via a command.
4. Inside the repository, create a directory named `.github/workflows`.
5. Create a YAML file within that directory (e.g., `ci.yml`) to define the workflow.

6. Configure the workflow with appropriate on triggers (e.g., `push` to the `main` branch).
7. Define the job(s) with a runner environment (e.g., `ubuntu-latest`).
8. Add steps to the job:
 - (a) Check out the repository code using `actions/checkout`.
 - (b) Set up the required language environment (e.g., `actions/setup-node`).
 - (c) Install project dependencies (e.g., `npm install` or `pip install -r requirements.txt`).
 - (d) Run the automated tests (e.g., `npm test` or `pytest`).
9. Commit and push the YAML workflow file to the GitHub repository.
10. Navigate to the "Actions" tab on the GitHub repository to observe the workflow execution.
11. Verify that the workflow runs successfully on each push and that all tests pass.
12. Study Jenkins conceptually by exploring its architecture, plugin ecosystem, and pipeline capabilities through documentation and tutorials.
13. Document observations and compare the features of GitHub Actions and Jenkins.

CODE

GitHub Actions Workflow File (.github/workflows/ci.yml)

```
1 name: Node.js CI
2
3 on:
4   push:
5     branches:
6       - main
7   pull_request:
8     branches:
9       - main
10
11 jobs:
12   build:
13     runs-on: ubuntu-latest
14
15     strategy:
16       matrix:
17         node-version: [18, 20]
18
19     steps:
20       - name: Checkout Repository
21         uses: actions/checkout@v4
22
23       - name: Setup Node.js ${{ matrix.node-version }}
24         uses: actions/setup-node@v4
25         with:
26           node-version: ${{ matrix.node-version }}
27
28       - name: Install Dependencies
29         run: npm install
30
31       - name: Run Tests
32         run: npm test
33
34       - name: Display Success Message
35         run: echo "Build and tests completed successfully!"
```

Sample Application (app.js)

```
1 function add(a, b) {  
2     return a + b;  
3 }  
4  
5 function subtract(a, b) {  
6     return a - b;  
7 }  
8  
9 function multiply(a, b) {  
10     return a * b;  
11 }  
12  
13 module.exports = { add, subtract, multiply };
```

Sample Test File (app.test.js)

```
1 const { add, subtract, multiply } = require('./app');  
2  
3 test('Addition', () => {  
4     expect(add(2, 3)).toBe(5);  
5 });  
6  
7 test('Subtraction', () => {  
8     expect(subtract(10, 4)).toBe(6);  
9 });  
10  
11 test('Multiplication', () => {  
12     expect(multiply(3, 7)).toBe(21);  
13 });
```

package.json (Scripts Section)

```
1 "scripts": {  
2     "test": "jest"  
3 }
```

OUTPUT

The GitHub Actions workflow was executed successfully. The following observations were recorded:

- The repository was automatically cloned by the runner.
- Node.js version 18 and 20 were configured correctly across the matrix build.
- Dependencies were installed without any errors.
- All test cases executed successfully (3 out of 3 passed).
- The workflow completed successfully after each push to the `main` branch.
- The GitHub Actions dashboard displayed a green checkmark indicating a successful workflow run.
- The workflow execution time was approximately 45 seconds.
- Jenkins was studied and understood as a robust, plugin-extensible CI/CD automation server suitable for enterprise-grade pipelines.

RESULT

The CI/CD pipeline was successfully implemented using GitHub Actions. The workflow automatically built the project, installed dependencies, and executed the test suite whenever code was pushed to the repository. All tests passed, confirming the application's correctness. Additionally, Jenkins was explored as a powerful and flexible automation server that supports advanced pipeline creation, distributed builds, and extensive plugin-based integrations for CI/CD.

CONCLUSION

This experiment provided practical, hands-on knowledge of CI/CD automation using GitHub Actions. The automated workflow significantly reduced manual effort in building and testing code, while ensuring that integration issues were detected early. The conceptual study of Jenkins highlighted its role as an industry-standard automation tool that supports complex, enterprise-grade pipelines through its plugin ecosystem and Pipeline as Code capabilities. Understanding CI/CD practices and tools is essential for modern software development, enabling teams to deliver high-quality software rapidly, reliably, and with minimal manual overhead.